Steering Through the Trees: Physical Lie Vector Field Evasion of Tree Ensembles in Network Intrusion Detection

Quang-Vinh Dang[a,*]

[a] *School of Innovative and Computing Sciences, British University Vietnam, Hung Yen, Vietnam*

Abstract

Gradient-boosted trees dominate flow-based network intrusion detection, yet their worst-case robustness to schema-feasible evasion is rarely measured. Flow-Field Evasion (FFE) models packet-level actions as commuting generators on the flow manifold. After diagnosing FFE-v1, we introduce FFE-v2 (re-segmentation, multi-scale probing, data-driven window targets), which uses scores only. On three full, deduplicated corpora and three libraries, it matches or exceeds our greedy threshold-crossing control and FFE-v1 in all nine cells (significantly above the control in four); on XGBoost its Real ASR is 99.5%, 100.0% and 98.3%, against 57.6% for FFE-v1 and 21.1% for CAPGD on UNSW-NB15. It needs 28–38 queries (control: 2). CatBoost on CIC-IDS2017 (36.4%) is a ceiling set by immutable features. Feasibility is relative to a schema verifier, and chaff accounting matters on UNSW-NB15. As a defense, a monotone gate is OR-fused with a free classifier (hardened on five of nine cells). If backward features stay fixed, flagged flows stay flagged.

*Corresponding author

Email address: vinh.dq4@buv.edu.vn (Quang-Vinh Dang)

Against FFE-v1, CAPGD and a greedy white-box attacker, Real ASR is at most 8.8% on CSE-CIC-IDS2018 and CIC-IDS2017 and 1.0–32% on UNSW-NB15, one seed; FFE-v2 was not run against it. The single test evaluation is not a clean hold-out; five-seed runs are cleaner.

Keywords: Adversarial machine learning, Network intrusion detection, Gradient boosted decision trees, Evasion attacks, Problem-space constraints, Lie algebra, Adversarial training

---

1. Introduction

Modern network intrusion detection systems (NIDS) increasingly rely on machine learning over flow-based telemetry. At multi-gigabit rates, inspecting every packet payload is prohibitive, so sensors such as CICFlowMeter [1] aggregate bidirectional packet exchanges into tabular flow records of timing, length, header and flag statistics. Gradient-boosted decision trees (GBDTs), notably XGBoost [2], LightGBM [3] and CatBoost [4], are the dominant classifiers for these records [5, 6].

Near-perfect benchmark accuracy does not guarantee robustness: attackers control the malleable parts of their traffic, such as timing, padding, segmentation and window sizes, so a detector that relies on these features can be evaded without losing the exploit [7, 8, 9].

Evading a tree ensemble is hard for two reasons. First, the ensemble is piecewise constant, so $\nabla_x F(x) = 0$ for almost all x and gradient-based attacks receive no signal. Second, flow features obey conservation laws. Flow Bytes/s satisfies $x_8 = 10^6(x_4 + x_5)/x_6$, where x_4 and x_5 are the forward and backward byte totals and x_6 is the duration in microseconds. A perturbation

that changes the byte totals without updating the rate produces a phantom flow that protocol sanity checks reject [10, 7]. Existing attacks either ignore this coupling, as ZOO [11] and HopSkipJump [12] do (HopSkipJump yields no feasible evasive flow on the corpora of this study; ZOO keeps at most 25.2% Real ASR, the share of attacks that evade and satisfy the schema invariants), or need hundreds of black-box queries per flow [13, 14, 15], or depend on a surrogate [13, 16].

Our earlier attack, Flow-Field Evasion (FFE-v1), derived four packet-level actions as closed-form generators of a Lie cone on the flow manifold, so that every step preserves the invariants, and steered with score probes. It evaded XGBoost in 99.5%, 99.2% and 57.6% of the detected attacks of three full corpora, but a rigorous control exposed its limits: a greedy white-box attack in the style of leaf-tuple search [17], which reads the trees but projects every candidate onto the manifold, reached 90.2% on UNSW-NB15, and FFE-v1 evaded LightGBM on CIC-IDS2017 in only 35.1% and CatBoost on CSE-CIC-IDS2018 in only 59.3% of the flows. We diagnosed these gaps on a validation split, not on the test cohorts: the attack lacked a generator that raises the packet count without adding payload, used one fixed magnitude per generator and one hard-coded window target (64,240 B) that lies outside the support of the CIC-IDS2017 data, and measured progress on a saturating probability.

This article reports the repaired attack, FFE-v2, and a defense built on what the attack revealed. The main contributions are as follows.

- **Generators.** We extend the Lie-cone model by a re-segmentation generator (the same payload carried by more, smaller packets). The five

fields still commute and preserve the rate, mean-length, header and MTU invariants of the schema, although they are not linearly independent (re-segmentation lies in the span of chaff and padding, outside their nonnegative cone; Section 3, Supplementary Section S1).

- **Attack.** FFE-v2 probes every generator on a geometric ladder of magnitudes, draws window targets from the attacker's benign sample, measures progress in log-odds and refines the cheapest crossing; it needs only scores (Section 3.4). In a single evaluation on frozen settings it matches or exceeds our greedy threshold-crossing white-box control in all nine corpus-library cells (significantly above it in four) and is significantly above FFE-v1 in seven (Real ASR 99.5%, 100.0% and 98.3% on XGBoost against 99.5%, 99.2% and 57.6% for FFE-v1 and 88.5%, 89.5% and 21.1% for CAPGD); five-seed means agree (Section 5). The control is our own implementation (erratic across seeds on CIC-IDS2017 LightGBM); LT-Attack and Cascioli et al. were not run.

- **Honest accounting.** FFE-v2 uses 28–38 score queries (109 on CIC-IDS2017 CatBoost), against 2 for the control and about 3 for CAPGD with an uncounted surrogate, and more than FFE-v1 on the easy corpora; its operational cost on UNSW-NB15 exceeds that of FFE-v1; one cell (CatBoost on CIC-IDS2017, 36.4%) is a ceiling set by attacker-immutable features rather than an attack success. "Feasible" is relative to a schema verifier that does not check the header-plus-payload bound, and chaff probes are booked as L_0 bytes of the length column; a synthetic replay shows that UNSW-NB15 evasions depend on chaff

accounting (Section 6).

- **Defense.** We OR-fuse a monotone gate, a tree ensemble that is non-decreasing in forward packets, forward bytes, duration and header bytes and does not read the window, rates or mean length, with a free classifier (augmentation-hardened on five of nine cells; the undefended model on the others). If the attacker leaves the backward features and the port and protocol unchanged, a flow the gate flags stays flagged (Section 7, a certificate in the spirit of monotone classification [18]). Against FFE-v1 ($M = 5, 20$), CAPGD and a greedy white-box threshold-crossing attacker, Real ASR on the flows still detected falls to at most 8.8% on CSE-CIC-IDS2018 and CIC-IDS2017 and to 1.0–32% on UNSW-NB15, with validation accuracy, F1 and recall not falling by more than 0.5 point. We state where this fails: about 8% of CSE-CIC-IDS2018 flows are not flagged by the gate and remain evadable, the stand-alone gate costs accuracy, and FFE-v2 was not run against the hardened models.

Caveat: FFE-v2 was tuned on validation cohorts and evaluated once on the test cohorts, but the FFE-v1 failures that motivated it were first seen on those test cohorts, re-segmentation was motivated by the control's moves on the same corpora, and the window pool uses the attacker's benign training sample (tuning queries are not counted); the five-seed study (retrained models, fresh cohorts, no tuning) is the cleaner confirmation. The attacks are red teaming that estimates what a schema-constrained adversary with score access can reach; no real network was attacked.

2. Related Work and Problem Formulation

2.1. Related Work and State-of-the-Art Taxonomy

Early adversarial attacks on NIDS were transplanted from image classification [19, 20, 21, 22]. Applied to flow features they reported high success rates, but the perturbed vectors were not network flows: they assign fractional values to packet counts, modify immutable fields such as destination port, and break the conservation between byte totals and packet counts. The problem-space view corrects this. Pierazzi et al. [10] showed that an adversarial perturbation is meaningful only when it is realized as a transformation of the physical object, such as a binary or a packet sequence, and that the transformation must preserve functionality and survive the feature extractor. Surveys and audits [23, 24, 7] argue that success rates overstate the risk once realistic constraints are enforced. Constrained tabular attacks address feasibility only in part: CAA [13] combines a gradient-based attack (CAPGD) with the MOEVA genetic search [14], which handles constraints by penalties, and Traffic Manipulator [15] mutates PCAP files with particle swarm search over intervals, padding and chaff.

Recent work makes the realism requirement concrete for NIDS. Merzouk et al. [25] show that many published evasion attacks cannot be realized on live traffic, Sheatsley et al. [26], Debicha et al. [27] and Zhang et al. [28] craft domain-constrained, botnet or transferable flows, and El Shehaby et al. [29] score which flow features an attacker can perturb, the mutable/immutable split that FFE builds into its schema. Learned generators [30, 16] trade query cost for a training phase, and tabular benchmarks [31, 32, 33] stress the role of constraints in reported success rates. The tree-ensemble literature moved

toward faster and deployable attacks [34], evasion detection [35] and robust training [36, 37]. None combines closed-form coupled-invariant generators with score-only steering and a certified defense for the same threat model.

Exact methods exploit the piecewise-constant structure of trees. Kantchelian et al. [38] cast minimal evasion against tree ensembles as a mixed-integer linear program. The formulation is exact but NP-hard and accepts only linear constraints, so it cannot express the nonlinear rate relations of flow records. MaxSAT encodings of tree ensembles [39] and SAT-based explanations of decision trees [40] provide formal explanations and robustness checks, and LT-Attack [17] attacks tree ensembles through a discrete search over leaf tuples. These methods operate in unconstrained feature space and do not enforce coupled network invariants unless a projection is added (Supplementary Section S7).

On the defensive side, robust training of tree ensembles [41, 42, 43, 44] and adversarial retraining of neural NIDS [45] improve robustness within a feature-space threat model, and verification of tree ensembles can certify robustness for restricted structures [46, 47]. The closest idea to our certificate is monotonic classification for malware detection [18]: if the attacker can only add content, a score that is monotone in the added features cannot be lowered. We transfer this argument to flow records, where the difficulty is the coupling between features, which we handle by hiding the derived columns from the certified component (Section 7).

Supplementary Table S19 summarizes these paradigms. Methods that preserve physical feasibility either need hundreds of black-box queries per flow or depend on a surrogate; FFE steers along packet-level generators and

chooses the direction with score probes, and the white-box control of Supplementary Section S7 shows the same moves are reachable with tree access.

2.2. Valid Packet Streams, Flow Telemetry and the Twelve-Feature Schema

A packet stream $P = ((p_1, t_1), \ldots, (p_m, t_m))$ is an ordered sequence of packets with $0 \le t_1 \le \cdots \le t_m$. Only streams in the admissible set $\mathcal{P}_{\text{valid}}$ are considered: every packet carries valid IPv4, TCP and UDP checksums [48, 49, 50] and obeys the interface MTU, $\text{Len}_{\text{IP}}(p_i) \le 1500$ bytes. A flow is the bidirectional set of packets sharing a 5-tuple, split into forward (client-to-server) and backward packets, and a flow exporter such as CICFlowMeter [1] computes a deterministic, many-to-one mapping $\Phi : \mathcal{P}_{\text{valid}} \to \mathbb{R}^d$, $x = \Phi(P)$.

We use a standardized 12-dimensional schema of the core metrics that state-of-the-art models analyze on CSE-CIC-IDS2018 [1]: x_0, destination port; x_1, protocol (6 for TCP, 17 for UDP); x_2, forward packets; x_3, backward packets; x_4, forward length total (the exporter's Total Length of Forward Packets); x_5, backward length total; $x_6 = (t_m - t_1) \times 10^6$, duration in microseconds; x_7, initial forward TCP window ($0 \le x_7 \le 65535$, $x_7 \equiv 0$ for UDP); x_8, flow bytes per second; x_9, flow packets per second; x_{10}, mean forward packet length; and x_{11}, forward transport header bytes, at least the per-packet minimum header size times x_2. The coupled columns are fixed by

$$x_8 = \frac{10^6\,(x_4 + x_5)}{x_6}, \qquad x_9 = \frac{10^6\,(x_2 + x_3)}{x_6}, \qquad x_{10} = \frac{x_4}{\max(1, x_2)}. \tag{1}$$

The generators of Section 3 book a chaff probe as L_0 bytes of x_4 (the wire size of an empty segment), whereas a re-segmented packet moves existing bytes and leaves x_4 unchanged. This accounting is a modelling choice that matters empirically (Section 6).

2.3. Feature Invariants and the Flow Manifold

The indices $\mathcal{D} = \{0, \dots, 11\}$ fall into constraint classes derived from network semantics. $\mathcal{I}_{\text{imm}} = \{0, 1, 3, 5\}$ (port, protocol, backward packets and bytes) are fixed by the service and the server. $\mathcal{I}^{+}_{\text{mono}} = \{2, 4, 6, 11\}$ can only grow: an attacker may add packets, pad payloads or delay transmissions, but cannot remove what the exploit needs. The window obeys $0 \le x_7 \le 65535 \cdot \mathbb{I}_{x_1=6}$, and $\mathcal{I}_{\text{dep}} = \{8, 9, 10\}$ are the deterministic functions above. In addition, $x_{11} \ge x_2(20\,\mathbb{I}_{x_1=6} + 8\,\mathbb{I}_{x_1=17})$ and $0 \le x_4 \le 1500 x_2$.

The physical flow space is the discrete image of valid packet streams, $\mathcal{M}_{\text{flow}} = \{x \mid \exists P \in \mathcal{P}_{\text{valid}},\ \Phi(P) = x\}$, with integer packet counts, byte totals, header totals, window and duration (in microseconds, at most $1.1 \times 10^8\,\mu$s, below the 120-second exporter timeout). For continuous differential geometry we relax counts and duration to reals and obtain the *ambient flow manifold*

$$\begin{aligned}\widetilde{\mathcal{M}}_{\text{flow}} = \Big\{ x \in \mathbb{R}^{12} \;\Big|\; & x_0 \in [1, 65535],\ x_1 \in \{6, 17\},\ x_2 \ge 1,\ x_3, x_5 \ge 0, \\ & 0 < x_6 \le 1.1 \times 10^8,\ H_{\min} x_2 \le x_{11} \le H_{\max} x_2,\ 0 \le x_4 \le 1500 x_2, \\ & 0 \le x_7 \le 65535\,\mathbb{I}_{x_1=6},\ x_{10} = \tfrac{x_4}{x_2},\ x_8 = \tfrac{10^6(x_4+x_5)}{x_6},\ x_9 = \tfrac{10^6(x_2+x_3)}{x_6} \Big\} \end{aligned} \tag{2}$$

with $H_{\min} = 20\,\mathbb{I}_{x_1=6} + 8\,\mathbb{I}_{x_1=17}$ and $H_{\max} = 60\,\mathbb{I}_{x_1=6} + 8\,\mathbb{I}_{x_1=17}$. Continuous flows are formulated on $\widetilde{\mathcal{M}}_{\text{flow}}$, and discrete execution maps trajectories back to $\mathcal{M}_{\text{flow}}$ through the closed-form projection $\mathcal{P}_{\mathcal{M}}$ (Section 3.4).

2.4. Threat Model

The adversary starts from a malicious flow $x_{\text{orig}} \in \mathcal{M}_{\text{flow}}$ with $F(x_{\text{orig}}) \ge 0$ (probability of the malicious class at least 0.5) and seeks $x^* \in \mathcal{M}_{\text{flow}}$ with $F(x^*) < 0$. Two access levels are used. The *score-only* attacker queries the

malicious probability of the target and nothing else; this is the headline setting of FFE-v2. The *threshold* attacker additionally reads the split thresholds of the trees, which enables the optional snapping step (Section 3.4) and is also the setting of the projected white-box control, which reads the trees entirely. Every probe counts as one query; transfer from a surrogate is covered by CAPGD and Supplementary Section S11.

The transformed trace must keep the malicious semantics so that exploit delivery, credential cracking or service degradation still succeeds; we do not test that. Padding appends non-executable bytes (e.g. SSH ignore messages [51]), and dilation lowers the burst intensity of volumetric floods. The NIDS is an ingress biflow sensor and the adversary controls only the client endpoint, so it changes the forward features $x_2, x_4, x_6, x_7, x_{11}$ and the derived rates, while the server-generated x_3 and x_5 are immutable. Chaff is sent as pure ACKs, which a TCP endpoint does not acknowledge (RFC 9293 [49]), so $\Delta x_3 = \Delta x_5 = 0$. *Re-segmentation is different*: more forward data segments can elicit more server ACKs under delayed-ACK rules, which would raise x_3; the schema holds x_3 fixed, so the segmentation moves assume that the added segments do not change the backward direction. We flag this as a limitation of the schema-level feasibility notion.

The adversary minimizes the operational cost $\mathcal{C}(x^*, x_{\text{orig}}) = \sum_k c_k s_k$, where s_k is the amount of generator k applied and c_k its unit cost in latency, bandwidth and socket resources (Section 3.3).

3. The Flow-Field Evasion Framework

3.1. Geometry of the Flow Manifold and Its Generators

The physical flow set $\mathcal{M}_{\text{flow}}$ is a discrete lattice in $\mathbb{R}^{12}$. Its differential geometry is analyzed through the smooth relaxation $\widetilde{\mathcal{M}}_{\text{flow}}$ (Section 2.2), the zero set of the conservation residual

$$\Psi(x) = \begin{bmatrix} x_8 - 10^6(x_4 + x_5)/x_6 \\ x_9 - 10^6(x_2 + x_3)/x_6 \\ x_{10} - x_4/x_2 \end{bmatrix} = \mathbf{0} \tag{3}$$

subject to the bounds listed there. At points where the Jacobian J_Ψ has full rank 3, the tangent space is $\ker J_\Psi(x)$, so a physically realizable velocity satisfies the linearized conservation laws $\dot{x}_8 = \frac{10^6(\dot{x}_4+\dot{x}_5)}{x_6} - \frac{x_8}{x_6}\dot{x}_6$, $\dot{x}_9 = \frac{10^6(\dot{x}_2+\dot{x}_3)}{x_6} - \frac{x_9}{x_6}\dot{x}_6$ and $\dot{x}_{10} = \frac{\dot{x}_4 - x_{10}\dot{x}_2}{x_2}$.

The attacker controls five elementary degrees of freedom, each generating a one-parameter flow whose push-forward is a tangent vector field ($\vec{e}_k$ is the kth basis vector of $\mathbb{R}^{12}$, indexed from 0):

$$\begin{aligned} \vec{v}_{\text{time}} &= \vec{e}_6 - \tfrac{x_8}{x_6}\vec{e}_8 - \tfrac{x_9}{x_6}\vec{e}_9, \qquad \vec{v}_{\text{pad}} = \vec{e}_4 + \tfrac{10^6}{x_6}\vec{e}_8 + \tfrac{1}{x_2}\vec{e}_{10}, \\ \vec{v}_{\text{chaff}} &= \vec{e}_2 + L_0\vec{e}_4 + \tfrac{L_0 10^6}{x_6}\vec{e}_8 + \tfrac{10^6}{x_6}\vec{e}_9 + \tfrac{L_0 - x_{10}}{x_2}\vec{e}_{10} + H_0\vec{e}_{11}, \\ \vec{v}_{\text{seg}} &= \vec{e}_2 + \tfrac{10^6}{x_6}\vec{e}_9 - \tfrac{x_{10}}{x_2}\vec{e}_{10} + H_0\vec{e}_{11}, \qquad \vec{v}_{\text{win}} = \mathbb{I}_{x_1=6}\ \text{sat}\left(\tfrac{W_{\text{target}} - x_7}{\epsilon_{\text{win}}}\right)\vec{e}_7. \end{aligned} \tag{4}$$

Temporal dilation adds delay without bytes or packets, padding adds forward dummy bytes, chaff adds forward header-only probes, booked as L_0 bytes of the length column, and window morphing moves the client window toward a target W_{target}. The probe sizes are $L_0 = 40$, $H_0 = 20$ for empty TCP segments [49], $L_0 = 52$, $H_0 = 32$ with TCP timestamps [52], and $L_0 = 28$, $H_0 = 8$ for UDP [50]. The new generator, *re-segmentation* (an MSS clamp),

carries the same forward payload in more, smaller packets: the packet count and the header total grow by H_0 per packet, while the byte total and the duration are unchanged and the mean length falls. It is the chaff field with $L_0 = 0$, hence $\vec{v}_{\text{seg}} = \vec{v}_{\text{chaff}} - L_0\vec{v}_{\text{pad}}$ component-wise: the five fields commute but are *not* linearly independent, and re-segmentation lies in the span of chaff and padding but outside their nonnegative cone (it corresponds to chaff with negative padding). An integer move of K packets is exactly the time-K flow of this field ($x_{10} = x_4/(x_2 + K)$). A sixth generator, TCP option padding ($\vec{v}_{\text{hdr}} = \vec{e}_{11}$), is implemented but is not part of the frozen configuration. The window field is globally Lipschitz; the implementation uses its limit $\epsilon_{\text{win}} \to 0$, which lands exactly on W_{target}.

3.2. Commutativity, Invariants and the Lie Cone

Proposition 1 (Commutativity of the physical generators)**.** *Because the protocol $x_1 \in \{6, 17\}$ is immutable, $\widetilde{\mathcal{M}}_{flow}$ decomposes into a TCP and a UDP component on which $\mathbb{I}_{x_1=6}$, L_0 and H_0 are constant along trajectories (FFE-v2 fixes $H_0 = 32$ iff the mean forward header $x_{11}/x_2 \geq 26$ of the original flow; FFE-v1 re-evaluates it on the current flow, where injecting packets of header size H_0 moves that mean toward H_0 and never across the threshold). On each component the fields $\vec{v}_{time}, \vec{v}_{pad}, \vec{v}_{chaff}, \vec{v}_{seg}$ are C^∞, $\vec{v}_{win}$ is Lipschitz, and*

$$[\vec{v}_i, \vec{v}_j] \equiv \mathbf{0} \qquad \textit{for all } i, j \in \{time, pad, chaff, seg, win\}, \tag{5}$$

so the fields span an Abelian Lie algebra, of dimension 4 on TCP (3 on UDP) because $\vec{v}_{seg}$ is a linear combination of $\vec{v}_{chaff}$ and $\vec{v}_{pad}$. The same holds with $\vec{v}_{hdr}$.

The proof is an entry-wise check of the four new brackets involving $\vec{v}_{\text{seg}}$ on top of the six of FFE-v1 (Supplementary Section S1, Lemma S1); the window field acts only on x_7, which no other field reads or writes. The admissible cone is $\mathcal{K}_{\text{net}}(x) = \{\sum_k u_k \vec{v}_k(x) \mid u_k \geq 0\} \subset T_x \widetilde{\mathcal{M}}_{\text{flow}}$.

Theorem 1 (Invariant preservation). *Let $x_0 \in \widetilde{\mathcal{M}}_{flow}$ and $u \in L^\infty_{loc}(\mathbb{R}_{\geq 0}, \mathbb{R}^5_{\geq 0})$ satisfy the admissibility rules: $u_{pad} \leq (1500 - L_0)u_{chaff} + 1500u_{seg}$ whenever $x_4 = 1500x_2$; $u_{time} = 0$ whenever $x_6 = 1.1 \times 10^8$; and $u_{seg} = 0$ whenever $x_{10} \leq 1$ (a re-segmented packet carries at least one payload byte). Then the solution of $\dot{x} = \sum_k u_k \vec{v}_k(x)$, $x(0) = x_0$, exists, is unique, is global and stays in $\widetilde{\mathcal{M}}_{flow}$.*

Proof sketch. Differentiating Ψ along the flow gives $\dot{\Psi} = A(t)\Psi$ with $A = \operatorname{diag}\big(-\frac{u_{\text{time}}}{x_6}, -\frac{u_{\text{time}}}{x_6}, -\frac{u_{\text{chaff}}+u_{\text{seg}}}{x_2}\big)$ locally integrable, so $\Psi \equiv 0$ by Carathéodory's theorem [53]. All remaining bounds are affine and satisfy the Nagumo–Brezis condition [54, 55] on every face under the admissibility rules, which gives forward invariance, existence and uniqueness. The full proof, including the new case of $\vec{v}_{\text{seg}}$ at the MTU face and its effect on the header-total bound, is in Supplementary Section S1. Under discrete stepping the closed-form operator $\mathcal{P}_{\mathcal{M}}$ is a retraction (exact on feasible inputs, not a metric projection) that removes truncation drift and enforces integrality.

The theorem concerns the continuous relaxation and the schema, not side effects at a real endpoint (Section efsec:formulation).

3.3. Operational Cost

The unit costs are $c_1 = 10^{-4}$ per second of induced delay, $c_2 = 10^{-2}$ per byte of padding, $c_3 = 1.0$ per added packet and $c_4 = 10^{-3}$ per normalized

window displacement $|W_{\text{target}} - x_7|/65535$. A re-segmented packet is priced exactly like a chaff packet, and header-option bytes are not priced. Because the generators commute, the displacement on the interior of the manifold depends only on the integrated amounts $s_k = \int u_k \, dt \geq 0$, and the cost $\mathcal{C}(s) = \sum_k c_k s_k$ is linear and path-independent there (not at the boundary faces). The cost model is the one FFE steers by. The boundary $F(x) = 0$ of a tree ensemble is piecewise constant and non-convex, so the evasion problem, minimizing $\mathcal{C}$ subject to $F(x(T^*)) < 0$ and the dynamics above, is solved heuristically (Section 3.4).

3.4. Algorithmic Realization: FFE-v2

FFE-v1 took one fixed-size probe per generator and iteration (Supplementary Algorithm S1). Validation diagnostics (Supplementary Section S3) showed where that stalls: on UNSW-NB15 the score plateaus until splits on the packet count, mean length and packet rate are crossed jointly, which needs payload-free packets in magnitudes of tens to hundreds, and elsewhere the single window target lay on the wrong side of the model's borders. FFE-v2 changes five things (Algorithm 1); all other machinery, in particular the projection $\mathcal{P}_{\mathcal{M}}$ that follows every move (Supplementary Algorithm S2), is unchanged.

Multi-scale probing.. Every generator is probed on a geometric ladder of integer magnitudes $a_k = \min(a_0 r^k, a_{\max})$ that spans its whole feasible range: delay from 1 ms to the 110 s cap, padding from 16 B to the MTU headroom, one to 4,096 chaff or segmented packets. A plateau of the piecewise-constant score is therefore crossed within one iteration, and identical projected candi-

Algorithm 1 FFE-v2 (score-only)

Require: Score oracle $p(\cdot)$ of the target, flow $x_0 \in \mathcal{M}_{\text{flow}}$ with $p(x_0) \geq 0.5$, generators $\mathcal{G} = \{\text{time}, \text{pad}, \text{chaff}, \text{seg}, \text{win}\}$, ladder ratio $r = 6$, iteration budget $M = 8$, relax-after $R = 3$, refinement probes $B = 4$, window pool $\mathcal{W}$

Ensure: Flow $x^* \in \mathcal{M}_{\text{flow}}$ with $p(x^*) < 0.5$, or FAILURE

$x \leftarrow \mathcal{P}_{\mathcal{M}}(x_0)$; $z \leftarrow \operatorname{logit} p(x)$
for $t = 1$ **to** M **do**
  $\mathcal{C} \leftarrow \{\mathcal{P}_{\mathcal{M}}(\text{apply}(x, g, a)) : g \in \mathcal{G} \setminus \{\text{win}\},\ a \in \text{Ladder}_g(x)\} \cup \{\mathcal{P}_{\mathcal{M}}(x \text{ with } x_7{=}w) : w \in \mathcal{W}\}$ {duplicates removed}
  score all candidates (one query each)
  if some candidate has $p < 0.5$ **then**
    $(g, a, x') \leftarrow$ cheapest crossing candidate under $\mathcal{C}(x_0, \cdot)$; **break**
  end if
  $\Delta_c \leftarrow z - \operatorname{logit} p(c)$ for each candidate; keep those with $\Delta_c \geq 10^{-3}$
  select $\arg\max\ \Delta_c/(\Delta\mathcal{C}_c + 10^{-4})$ if $t \leq R$, else $\arg\max \Delta_c$ (also if no efficient candidate exists); **stop with FAILURE if none**
  $x \leftarrow$ selected candidate; $z \leftarrow \operatorname{logit} p(x)$
end for
if a crossing (g, a, x') was found **then**
  bisect a with B further probes of the parent state; keep the smallest magnitude that is still scored benign
  return x^*
end if
return FAILURE

dates are probed once.

Data-driven window targets.. The window generator tries the six most frequent benign TCP windows of the attacker's benign sample (the training split), in both directions, instead of 64,240 B. A variant with generic operating-system defaults (8,192; 14,600; 29,200; 64,240; 65,535) needs no data.

Log-odds progress and selection.. Progress is the decrease of the log-odds of the returned score, a monotone transform of the same score, which does not saturate where float32 probabilities near 1 tie. If any probe crosses 0.5, the cheapest crossing under the operational cost of Section 3.3 ends the search;

otherwise the move with the largest decrease per unit of marginal cost is taken, switching to pure score minimization after three iterations or when no move is cost-efficient.

Cost refinement.. After a crossing, four bisection probes shrink the magnitude of the crossing move; each probe is a scored flow, so success is never assumed.

Optional snapping (Tier 1).. When no probe helps, threshold crossings of *all* mutable features (eight per feature and direction) are converted to closed-form generator magnitudes, including exact inversions of the rate and mean thresholds through delay, padding, chaff and segmentation. This is the only use of the trees' thresholds; the headline variant does not snap.

Query accounting and configuration.. Every scored flow, including probes scored in a batch, is one query. Queries are bounded by the iteration budget times the ladder size plus the refinement probes; observed means are 28–38 on every cell except CIC-IDS2017 CatBoost (109), where it exhausts its budget on unreachable flows (Section 6). All hyper-parameters (ladder ratio, number of levels, window pool size, refinement probes, iteration budget, relax rule) were chosen on a validation split over 58 configurations (Supplementary Table S1) and frozen before the single test evaluation: the ladder ratio, refinement probes and window pool size are in `results/ffev2_config.json`, and the iteration budget $M = 8$, the relax rule $R = 3$ and the ladder range are the defaults of `src/attacks/ffe_v2.py`. Beam search and the option-padding generator gave no gain worth their queries and were not kept.

4. Experimental Protocol

4.1. Corpora, Targets and Attackers

Single-testbed evaluations have limited external validity [56], and attack evaluations that ignore realism overstate the risk [7, 23], so we use three widely studied corpora in full: CSE-CIC-IDS2018 [1] (nine daily files, seven capped at 1,048,575 rows in the distribution), CIC-IDS2017 [1] (five daily files) and UNSW-NB15 [57] (the official training and testing partition, 257,673 of the 2.54 million records). Every record is mapped onto the 12-feature schema of Section efsec:formulation, invalid records are dropped, and every flow is projected onto the manifold *before training and testing* (Supplementary Section S12); this pre-projection clamps byte totals above 1500 times the packet count, durations above 110 s and UDP header sizes, so the learned models never see such values. Records with identical features and label are then collapsed so that no flow appears on both sides of the split [58, 59], which removes 39.1%, 20.5% and 42.7% of the valid records and leaves 4,968,389, 1,658,869 and 123,508 flows (Supplementary Table S18). Records with identical features but different labels (149,526, 22 and 1,040) are kept as real ambiguity of the schema. The split into training, validation and test partitions is 70%/10%/20%, stratified by label and family, random, and not grouped by time or host, so near-duplicates may fall on both sides and accuracies are optimistic.

The targets are XGBoost (CUDA), LightGBM (each 60 trees of depth 6, learning rate 0.1) and CatBoost (60 oblivious trees of depth 6). *FFE-v2* is score-only; its snapping variant also reads thresholds. *FFE-v1* (Supplementary Algorithm S1) is the reference baseline and reads thresholds for its

snapping step. The *projected white-box control* reads all trees: each step moves one feature across one split threshold, choosing the move that lowers the exact ensemble margin most, with every candidate projected onto the manifold before scoring; it counts two queries (the tree reads are free) and is cost-blind. For XGBoost it is a greedy leaf-tuple-style attack in the spirit of Zhang et al. [17]; for LightGBM and CatBoost it is our generic version on the raw ensemble margin. It is not LT-Attack (whose official code needs a Bazel/C++ build we did not set up) and we did not run Cascioli et al. [34]. On XGBoost we also compare with seven earlier baselines: a naive coordinate descent, *ZOO-NIDS* [11], *HopSkipJump-NIDS* [12], *MOEVA* [13, 14], *PSO* [15] with our projection, and our re-implementations of *CAPGD* [13] and a *NIDSGAN-style generator* [16], which fit a small MLP surrogate on the training flows (surrogate computations are not counted). CAPGD was run on XGBoost in the single-split test and, in the seed study, also on LightGBM and CatBoost (three seeds); baselines use default budgets.

4.2. Protocol, Metrics and Cohorts

All design choices and hyper-parameters of FFE-v2 were tuned (tuning queries are not counted) on cohorts of up to 1,000 detected malicious flows from the 10% *validation* split; the configuration was frozen and evaluated once on the benchmark's 1,000-flow *test* cohorts (the identical flows used for FFE-v1 and the baselines, verified by the runner). This is not a clean hold-out: the FFE-v1 failures that motivated FFE-v2 were first observed on these test cohorts, re-segmentation was motivated by the control's moves on the same corpora, and the window pool uses the attacker's benign training sample. The five-seed study (retrained models, fresh cohorts, no tuning)

is the cleaner confirmation. The cohorts are drawn once, with a fixed seed, from the malicious test flows that both XGBoost and LightGBM detect when clean (77.6%, 99.8% and 87.0% of the test attacks of CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15), so all results are conditional on detection. For CatBoost the cohort is restricted to the flows CatBoost itself detects ($N = 1000$, 997 and 983), so every success is a genuine evasion of that model. MOEVA and PSO need 183–445 queries per flow and attack the first 200 flows. FFE-v1 on XGBoost and LightGBM and the baselines are the saved benchmark outcomes; FFE-v1 on CatBoost and the control on LightGBM and CatBoost were run for this study.

ASR is the fraction of attacks with $P(\text{Malicious} \mid x^*) < 0.5$; PFR the fraction of evasive candidates that satisfy the schema invariants; Real ASR counts an attack only when it both evades and is feasible. Feasibility is the schema verifier of Section 2.2 (immutability, monotonicity, MTU and header bounds, 5% tolerance on the coupled rates, identical for every method), which holds for FFE by construction; it does not check the wire-byte gap $x_4 \geq x_{11} + 20x_2$, which re-segmentation would violate if x_4 counted headers. "100% schema-feasible" is relative to this verifier. We report Wilson 95% intervals and exact McNemar tests on paired outcomes; no multiplicity correction is applied except where stated (nine cells, so individual p-values should be read with that in mind), and pooled seed tests treat flows as independent. Cost is the median over a method's own feasible evasions of the operational cost of Section 3.3 and of the normalized distance D_{L_1}; these medians cover different flows, so we add paired medians on flows two methods both evade. For the multi-seed study every seed retrains the targets on a new stratified split and

redraws a cohort of up to 300 flows detected by XGBoost (LightGBM and CatBoost are attacked on the part of that cohort they also detect); the frozen configuration is used without tuning, with five seeds for XGBoost and three for LightGBM and CatBoost.

5. Attack Results

5.1. Benchmark Against XGBoost

Table 1 compares FFE-v2 with FFE-v1, the projected white-box control and the seven earlier baselines. Evading the model in feature space is not the same as producing a feasible flow: HopSkipJump-NIDS reaches 100.0% ASR on two corpora yet none of its evasive candidates is feasible, and the naive attack and ZOO-NIDS keep only 34.4% and 35.0% of their evasions on CIC-IDS2017 and 0.9% and 0.0% on UNSW-NB15. On CSE-CIC-IDS2018 they keep 88.0% and 83.2% at essentially zero cost, which is consistent with moving only coupled rate features within the 5% tolerance of the consistency check, so that check flatters them.

FFE-v2 reaches a Real ASR of 99.5%, 100.0% and 98.3% on CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15. It is significantly above all seven earlier baselines on every corpus (21 exact McNemar tests, largest Holm-adjusted $p = 8 \times 10^{-28}$); the closest competitor, CAPGD (88.5%, 89.5%, 21.1%), is beaten on 110, 105 and 772 flows against none. Against the stronger references the picture is more precise. On the two easy corpora our greedy white-box control, which reads the trees, ties FFE-v2 (99.5% and 100.0%, identical flows); the five CSE-CIC-IDS2018 flows that FFE-v2, FFE-v1 and the control all miss are DDoS flows that no tested move reached. On

Table 1: Evasion benchmark against CUDA XGBoost on the full corpora. All methods except MOEVA and PSO attack the same 1,000 detected malicious test flows; MOEVA and PSO attack the first 200 of them. ASR, PFR and Real ASR in %; PFR is the share of evasive candidates that are schema-feasible; Op. cost and D_{L_1} are medians over the method's own feasible evasions; – means no feasible evasion. FFE-v1 reads thresholds, the control reads the trees, FFE-v2 (bold) reads scores only.

Method	ASR	PFR	Real ASR [95% CI]	Queries	Op. cost	D_{L_1}	ms/flow
CSE-CIC-IDS2018 (XGBoost accuracy 97.30%, test attacks 118,963, detected by both models 92,275)							
Unconstrained feature space	36.8	88.0	32.4 [29.6, 35.4]	66.8	0.000	0.000	48.0
ZOO-NIDS [11]	30.3	83.2	25.2 [22.6, 28.0]	116.7	0.000	0.000	91.0
HopSkipJump-NIDS [12]	100.0	0.0	0.0 [0.0, 0.4]	47.0	–	–	30.3
MOEVA (search stage of CAA) [13, 14]	40.5	70.4	28.5 [22.7, 35.1]	330.1	0.092	0.001	253.2
PSO (Traffic Manipulator, projected) [15]	54.0	100.0	54.0 [47.1, 60.8]	182.9	16.3	0.061	142.0
CAPGD (surrogate) [13]	88.5	100.0	88.5 [86.4, 90.3]	3.4	46.1	0.824	15.4
NIDSGAN-style [16]	61.3	100.0	61.3 [58.2, 64.3]	2.5	48.0	0.836	7.1
FFE-v1 (reads thresholds)	99.5	100.0	99.5 [98.8, 99.8]	16.5	0.001	0.116	15.8
White-box control (projected, reads trees)	99.5	100.0	99.5 [98.8, 99.8]	2.0	0.000	0.120	19.1
FFE-v2 + snapping (reads thresholds)	99.5	100.0	99.5 [98.8, 99.8]	30.8	0.000	0.006	36.0
FFE-v2 (score-only)	99.5	100.0	**99.5 [98.8, 99.8]**	30.3	0.000	0.006	44.6
CIC-IDS2017 (XGBoost accuracy 99.96%, test attacks 79,281, detected by both models 79,150)							
Unconstrained feature space	64.8	34.4	22.3 [19.8, 25.0]	37.6	0.000	0.000	30.6
ZOO-NIDS [11]	62.2	35.0	21.8 [19.4, 24.5]	79.9	0.000	0.000	58.5
HopSkipJump-NIDS [12]	100.0	0.0	0.0 [0.0, 0.4]	47.0	–	–	36.5
MOEVA (search stage of CAA) [13, 14]	64.0	59.4	38.0 [31.6, 44.9]	225.9	0.059	0.005	201.0
PSO (Traffic Manipulator, projected) [15]	35.0	100.0	35.0 [28.7, 41.8]	251.4	15.2	0.137	203.3
CAPGD (surrogate) [13]	89.5	100.0	89.5 [87.4, 91.3]	3.2	15.0	0.305	15.1
NIDSGAN-style [16]	65.3	100.0	65.3 [62.3, 68.2]	2.4	24.6	0.490	5.8
FFE-v1 (reads thresholds)	99.2	100.0	99.2 [98.4, 99.6]	14.6	0.001	0.289	10.7
White-box control (projected, reads trees)	100.0	100.0	100.0 [99.6, 100.0]	2.0	0.000	0.045	11.9
FFE-v2 + snapping (reads thresholds)	100.0	100.0	100.0 [99.6, 100.0]	28.3	0.001	0.187	26.5
FFE-v2 (score-only)	100.0	100.0	**100.0 [99.6, 100.0]**	28.3	0.001	0.187	37.1
UNSW-NB15 (XGBoost accuracy 88.51%, test attacks 11,113, detected by both models 9,673)							
Unconstrained feature space	21.6	0.9	0.2 [0.1, 0.7]	48.5	0.082	0.000	49.1
ZOO-NIDS [11]	20.6	0.0	0.0 [0.0, 0.4]	130.1	–	–	96.5
HopSkipJump-NIDS [12]	8.1	0.0	0.0 [0.0, 0.4]	47.0	–	–	40.6
MOEVA (search stage of CAA) [13, 14]	13.0	100.0	13.0 [9.0, 18.4]	445.0	0.092	0.002	339.2
PSO (Traffic Manipulator, projected) [15]	42.0	100.0	42.0 [35.4, 48.9]	228.4	21.6	0.060	164.3
CAPGD (surrogate) [13]	21.1	100.0	21.1 [18.7, 23.7]	16.2	53.2	1.158	73.4
NIDSGAN-style [16]	11.3	100.0	11.3 [9.5, 13.4]	4.5	34.0	1.211	11.2
FFE-v1 (reads thresholds)	57.6	100.0	57.6 [54.5, 60.6]	93.7	5.610	0.027	89.1
White-box control (projected, reads trees)	90.2	100.0	90.2 [88.2, 91.9]	2.0	213.0	0.170	26.4
FFE-v2 + snapping (reads thresholds)	98.5	100.0	98.5 [97.5, 99.1]	37.9	36.0	0.121	81.8
FFE-v2 (score-only)	98.3	100.0	**98.3 [97.3, 98.9]**	36.2	40.0	0.121	68.0

UNSW-NB15 FFE-v2 exceeds the control (98.3% against 90.2%; 89 flows against 8, $p = 2 \times 10^{-18}$) and FFE-v1 (57.6%; 412 against 5). FFE-v1 had been 32.6 points *below* the control there, and the difference is the repair described in Section 3.4: on the validation flows that FFE-v1 missed, the

control's successful moves raise the packet count by a median of 213 and in 82% of the cases leave the byte total unchanged, which is the re-segmentation move that FFE-v1 could not express (Supplementary Section S3). FFE-v2 with snapping, which reads thresholds, is statistically indistinguishable on XGBoost (98.5% on UNSW-NB15; $p = 0.5$), so the score-only variant is the headline.

The methods pay differently. CAPGD and the NIDSGAN-style generator need far fewer target queries (3.2–16.2 and 2.4–4.5) because their gradient work runs on an uncounted surrogate, and the control needs two. FFE-v2 averages 30.3, 28.3 and 36.2 queries, about twice FFE-v1 on the easy corpora (16.5 and 14.6) and well below it on UNSW-NB15 (93.7). On the same 200 flows it needs 6 to 12 times fewer queries than PSO and MOEVA, although query counts are not strictly like-for-like across score, decision and surrogate methods. The median operational cost of FFE-v2's feasible evasions is 3.0×10^{-5}, 5.5×10^{-4} and 40.0 units, against 5.7×10^{-4}, 8.6×10^{-4} and 5.6 for FFE-v1; on UNSW-NB15 this partly reflects who succeeds (FFE-v1 evades only the cheap flows; Section 6), and the median of 40 is dominated by tens to hundreds of chaff or re-segmented packets priced at one unit each. These comparisons are specific to the cost model, which FFE steers by.

5.2. Three Libraries and Three Corpora

Table 2 extends the comparison to LightGBM and CatBoost. FFE-v2 is at least as successful as our greedy projected control and as FFE-v1 in all nine cells. It is significantly higher than the control in four (UNSW-NB15 for all three libraries, $p \leq 5 \times 10^{-15}$, and CIC-IDS2017 LightGBM, 594 flows against none) and than FFE-v1 in seven (exact McNemar; the weakest is

Table 2: Real ASR (%) on the 1,000-flow test cohorts (CatBoost: flows it detects), per corpus and target library, with the clean test accuracy of the target and mean queries per flow (q). Single frozen evaluation. FFE-v1 and +snap read thresholds, the control reads the trees, FFE-v2 reads scores only.

Corpus	**Target**	N	**Acc.**	**FFE-v1**	**Control**	**FFE-v2**	**+snap**	q **v1**	q **v2**
CSE-CIC-IDS2018	XGBoost	1000	97.30	99.5	99.5	**99.5**	99.5	16.5	30.3
	LightGBM	1000	97.28	99.5	99.5	**99.5**	99.5	20.2	31.1
	CatBoost	1000	97.26	59.3	99.5	**99.5**	99.5	52.5	29.1
CIC-IDS2017	XGBoost	1000	99.96	99.2	100.0	**100.0**	100.0	14.6	28.3
	LightGBM	1000	99.96	35.1	36.6	**96.0**	96.0	120.1	32.4
	CatBoost	997	99.85	34.8	36.4	**36.4**	36.4	76.3	109.0
UNSW-NB15	XGBoost	1000	88.51	57.6	90.2	**98.3**	98.5	93.7	36.2
	LightGBM	1000	88.47	62.4	88.1	**96.7**	99.2	89.0	38.3
	CatBoost	983	86.67	70.1	56.8	**97.5**	97.7	67.9	38.0

CIC-IDS2017 XGBoost, 8 flows against none, $p = 0.008$); the remaining cells tie with identical flows. The two failure modes of FFE-v1 are repaired. On CSE-CIC-IDS2018 CatBoost FFE-v1 evaded 59.3% because its single window target (64,240 B) lies on the wrong side of CatBoost's borders, whereas on the validation cohort the control evades 411 of FFE-v1's 414 failures with a window move alone, and FFE-v2 reaches 99.5%. On CIC-IDS2017 LightGBM FFE-v1 reached 35.1% for the same reason: 64,240 B almost never occurs in the CIC-IDS2017 training flows (0.08% of benign and 0.015% of attack TCP flows), so each ensemble places its splits arbitrarily in that near-empty interval (XGBoost routes it to the benign side, LightGBM keeps it malicious; Supplementary Section S9). Drawing the targets from the attacker's benign sample lifts LightGBM to 96.0%, and the residual is concentrated in DoS flows (90.5% evaded), which we did not examine further.

A ceiling, not a win.. CatBoost on CIC-IDS2017 stays at 36.4% for FFE-v2 and the control (34.8% for FFE-v1). The decision for DDoS and DoS flows rests on attacker-immutable features (backward packets, backward bytes and

destination port carry 77% of CatBoost's gain), and a 20,000-sample multi-start search over all generators found no crossing for any of 37 sampled DDoS and 40 sampled DoS validation flows (3 of 80 sampled failed flows were reachable, all port scans; `results/ffev2_report.md`). FFE-v2's own Real ASR on these families is 0.0% (DDoS) and 5.0% (DoS). We found no crossing in a multi-start search, which cannot prove unreachability; the number describes the model's reliance on immutable fields, not attack strength, and the unreachable flows make FFE-v2 spend its whole budget (109 queries).

Snapping and per-family results.. Threshold snapping adds up to 2.5 points on UNSW-NB15 (LightGBM, 96.7% to 99.2%, $p = 6 \times 10^{-8}$) and at most 0.2 point elsewhere. Per-family rates are in Supplementary Table S5; on UNSW-NB15 every XGBoost family with more than 20 flows is above 90%.

5.3. Retraining Variance

Table 3: Real ASR (%, mean ± standard deviation over seeds) of the frozen attacks; each seed retrains the target and draws a new cohort of up to 300 detected flows. Five seeds for XGBoost, three for LightGBM and CatBoost. q: mean queries of FFE-v2.

Corpus	Target	Seeds	FFE-v2	FFE-v1	Control	CAPGD	q (v2)
CSE-CIC-IDS2018	XGBoost	5	**99.7 ± 0.3**	99.7 ± 0.3	99.7 ± 0.3	97.7 ± 2.4	31.1
	LightGBM	3	**99.7 ± 0.3**	99.7 ± 0.3	99.7 ± 0.3	98.9 ± 1.1	31.2
	CatBoost	3	**99.7 ± 0.3**	56.4 ± 4.3	99.7 ± 0.3	94.4 ± 2.5	29.2
CIC-IDS2017	XGBoost	5	**100.0 ± 0.0**	97.3 ± 1.5	100.0 ± 0.0	82.4 ± 11.9	29.1
	LightGBM	3	**95.6 ± 0.2**	36.1 ± 4.2	77.4 ± 31.6	50.0 ± 2.4	30.3
	CatBoost	3	**36.7 ± 3.5**	35.1 ± 3.1	36.5 ± 3.3	23.0 ± 2.8	122.1
UNSW-NB15	XGBoost	5	**97.7 ± 2.5**	58.3 ± 4.3	86.0 ± 8.0	31.5 ± 2.8	38.4
	LightGBM	3	**98.9 ± 1.0**	57.2 ± 5.5	90.9 ± 8.2	30.4 ± 4.9	37.0
	CatBoost	3	**99.3 ± 0.3**	72.1 ± 0.9	74.4 ± 5.6	30.1 ± 6.2	37.2

A single split and cohort cannot show how much of the ranking is luck. Table 3 repeats the comparison without any tuning. FFE-v2 is never below

FFE-v1, the control or CAPGD in any seed of any cell, and strictly above them in most. On UNSW-NB15 XGBoost its mean is 97.7% against 58.3% (FFE-v1), 86.0% (control) and 31.5% (CAPGD); pooled over the five seeds it evades 188 flows that the control misses and misses 12 that the control evades ($p = 8 \times 10^{-42}$). The control, our own implementation, is erratic where the model depends on a single move: on CIC-IDS2017 LightGBM its Real ASR is 95.7%, 41.0% and 95.7% over the three seeds (mean 77.4%), whereas FFE-v2 holds 95.6% $\pm$ 0.2. CatBoost on CIC-IDS2017 remains at the ceiling (36.7% $\pm$ 3.5). Pooled McNemar tests and win–tie–loss counts are in Supplementary Table S6. Three seeds for LightGBM and CatBoost are few, pooled p-values treat flows as independent, and nine cells are tested without multiplicity correction. The seeds vary the split and the cohort but the configuration was tuned on the main split's validation cohorts; they are the cleaner confirmation, not an independent tuning test.

6. What Drives the Attack: Ablations, Cost and Packet-Level Replay

Ablations on the validation split.. Supplementary Table S1 varies one factor at a time over nine cells. The data-driven window pool is decisive on the CSE-CIC-IDS2018 and CIC-IDS2017 corpora (removing it drops mean Real ASR from 91.6% to 60.6%), chaff matters on UNSW-NB15 for LightGBM and CatBoost (59.9% and 63.9% without it), and the ladder must span the range. Re-segmentation adds little on validation once chaff and padding exist (96.6%, 96.9% and 97.2% without it against 96.7%, 97.5% and 97.4% on UNSW-NB15), yet 39%, 16% and 24% of FFE-v2's successful UNSW-

NB15 flows use it (Supplementary Table S4).

Cost.. FFE-v2 is not uniformly cheaper than FFE-v1: on UNSW-NB15 its paired median operational cost on the flows both evade is higher (6.7 against 5.8, 14.6 against 7.5 and 29.1 against 9.8 for XGBoost, LightGBM and CatBoost), and against the control it is cheaper for XGBoost and LightGBM (32.5 against 213 and 35.8 against 49) but not for CatBoost (22.1 against 10.5; Supplementary Tables S2 and S3).

Packet-level replay.. To check that the generators correspond to realizable packet operations we expanded the action ledger of each successful flow of the first 300 test flows per corpus (XGBoost) into a synthetic packet trace, re-extracted the twelve features with a CICFlowMeter-style extractor (our re-implementation) and re-scored the target (Supplementary Section S8, Table S13). When chaff packets carry the booked bytes, or empty chaff is compensated by padding, 100% of the replayed flows reproduce the schema vector within 1%, stay feasible and stay evasive, including the 118 UNSW-NB15 flows that use re-segmentation (replayed ASR 99.3%, 100.0% and 98.3%). If chaff packets are empty and the bytes the schema credits to them are not sent, the easy corpora are unchanged but UNSW-NB15 retains only 69.2% of its evasions (replayed ASR 68.0%), the same sensitivity as for FFE-v1. The replay validates the action semantics and the schema's internal consistency only: the traces are synthetic, no TCP state machine or server reaction is modelled (in particular whether more forward segments elicit more server ACKs), and nothing was sent over a network.

7. A Monotone-Gate Defense with a Conditional Certificate

The attacks suggest what a defender can and cannot promise. The schema lets the attacker only *increase* forward packets, forward bytes, duration and header bytes, hold the port, protocol and backward direction fixed, and move the window, the two rates and the mean length freely or as functions of the others. A score that never decreases when those four columns increase, that does not read the free or derived columns, and whose other inputs stay fixed, cannot be lowered by a feasible move. This section builds a defense on that observation, evaluates it, and states where it stops. The monotone-model idea is due to Incer et al. [18]; what is new here is its use on coupled flow records, where the derived columns must be hidden.

7.1. Design

Component 1: monotone gate.. A tree ensemble that (i) is never shown the window, the two rates and the mean length (x_7, x_8, x_9, x_{10}), and (ii) carries monotone constraints (non-decreasing malicious score) on the four increase-only columns x_2, x_4, x_6, x_{11}, with larger capacity (300 trees of depth 8, against 60 of depth 6 for the undefended model) to offset the constraint. The gate still reads the port, the protocol and the backward packet and byte totals (x_0, x_1, x_3, x_5), on which it is not monotone. Constraints are enforced by the library (XGBoost, LightGBM and CatBoost monotone constraints) and checked by a randomized test that pushes feasible transformations through gates of all libraries and asserts that the margin never falls; a companion test shows that an unconstrained model does fall, so the check is not vacuous.

Proposition 2 (Conditional certificate)**.** *Let G be non-decreasing in x_2, x_4, x_6, x_{11} and independent of $x_7, \ldots, x_{10}$, and let x' be derived from a flow x by actions that leave x_0, x_1, x_3, x_5 unchanged and do not decrease x_2, x_4, x_6, x_{11} (up to the verifier's absolute tolerance of 10^{-2}, below the integer resolution of these columns). Then $G(x') \geq G(x)$, so $G(x) \geq \tau$ implies $G(x') \geq \tau$ and the fused score $\max(free, G)$ of x' stays above its alert threshold.*

The proof is immediate from the definition of monotonicity (Supplementary Section S1) and does not depend on the generators. The assumptions are strong and we state them explicitly. (i) *The backward features x_3, x_5 and the port and protocol do not change.* The gate reads them and is not monotone in them, and chaff, padding, delay and especially re-segmentation can change server responses (more segments can elicit more ACKs), so the certificate does not cover attacks that move the backward direction. A follow-up experiment that hides x_3 and x_5 from the gate is reported in the supplementary when finished *[placeholder: results pending]*. (ii) The attacker cannot remove packets or bytes, split or merge flows, or exploit the feature extractor. (iii) The detector reads the twelve schema features as exported. (iv) The certificate holds if the library's monotone constraint holds, which we verified by randomized test, not by proof. (v) It is relative to the attacker's chosen original flow: a lower-footprint variant of the same attack flow (fewer packets, bytes or less duration) is a different original flow and is not covered, so the certificate is not a worst-case bound on an attack. (vi) It protects the flows the gate flags; it says nothing about flows it does not flag, about the false-positive rate or about poisoning.

Component 2: free classifier and fusion.. Three options were tried on validation: *orbit* training (invariance by augmentation with 100,000 randomly transformed copies of flows of both classes that keep their true labels), a lighter variant, and iterative *adversarial training* on flows crafted by FFE-v1 ($M = 5, 20$) and the white-box attacker (Supplementary Section S6). The fused score is max(free, gate shifted to fire at τ). For each library and corpus the pair (free classifier, $\tau \in \{\text{none}, 0.5, 0.8, 0.95, 0.99\}$) is chosen on the validation split only: among pairs whose validation accuracy, F1 and recall do not fall by more than 0.5 point below the undefended model, we take the lowest mean validation Real ASR of FFE-v1 ($M = 20$) and the white-box attacker on a 300-flow cohort. This selection is **V2**; on four of the nine cells it picks the undefended model as free component, so the gain there comes from the gate alone (Supplementary Table S11). **V2-R** allows accuracy and F1 to fall by up to 3 points; it was defined after seeing the first results and differs from V2 only on UNSW-NB15. The earlier defenses are kept as baselines: *MPS* hides duration and the two rates, and *LGAT* augments training with 5,000 FFE-v1 flows at $M = 5$ (Supplementary Algorithm S4).

7.2. Protocol

The cohort is 1,000 random malicious test flows detected by the undefended model of the same library (cohorts differ across libraries); each model is attacked on the cohort flows it still detects, and Real ASR is computed on those. The attackers are FFE-v1 at $M = 5$ and $M = 20$, CAPGD with a surrogate fitted to the *defended* model's soft outputs, and an adaptive white-box threshold-crossing attacker that reads the defended trees including the gate (a generalization of the control of Section 4 to any library, which also

tries one step either side of each threshold). *FFE-v2 was not run against the hardened models*, since they were not persisted and retraining takes hours; the gate is certified against it by Proposition 2, but the free component is not. The free classifiers trained on FFE-v1 and white-box flows are evaluated with the same attack code, so part of the protection is against the training attacks. One seed and one split per cell; Wilson intervals and McNemar tests cover flow sampling only.

7.3. Results

Table 4: Defenses against the adaptive attackers (Real ASR, %) per corpus and library. U: undefended; LGAT: Lie-guided adversarial training; V2: gate fused with the hardened classifier (bold). FFE: FFE-v1 with $M = 20$; WB: white-box threshold-crossing attacker. Acc. and Rec. are clean test accuracy and attack recall (%) of the undefended model and V2. Real ASR is computed on the flows the model still detects; whole-cohort retention is in Supplementary Tables S8–S10. Cohort of 1,000 detected flows per cell; one seed.

Corpus	Library	Acc. U/V2	Rec. U/V2	Undefended			LGAT			V2		
				FFE	CAPGD	WB	FFE	CAPGD	WB	FFE	CAPGD	WB
CSE-CIC-IDS2018	XGBoost	97.30/97.29	77.6/78.4	99.5	99.0	99.5	73.1	96.1	99.5	**8.1**	**0.3**	**8.1**
	LightGBM	97.28/97.27	77.8/78.2	99.8	99.5	99.8	56.8	96.3	99.8	**7.9**	**0.2**	**7.9**
	CatBoost	97.26/97.25	77.6/77.9	59.8	99.3	100.0	45.6	83.8	100.0	**0.0**	**0.0**	**8.8**
CIC-IDS2017	XGBoost	99.96/99.97	99.8/99.9	99.1	77.0	100.0	31.9	52.6	100.0	**0.7**	**0.7**	**0.7**
	LightGBM	99.96/99.97	99.9/99.9	34.6	57.9	35.7	33.4	36.3	35.6	**0.4**	**0.4**	**0.4**
	CatBoost	99.85/99.93	99.4/99.9	32.0	5.4	33.6	31.2	31.0	33.2	**0.1**	**0.1**	**0.1**
UNSW-NB15	XGBoost	88.51/88.38	88.0/90.4	62.2	24.4	90.9	15.0	6.2	81.7	**12.2**	**1.6**	**25.4**
	LightGBM	88.47/88.09	87.9/90.5	66.4	32.7	89.5	14.5	6.3	72.3	**22.0**	**2.1**	**31.9**
	CatBoost	86.67/86.42	89.6/92.6	72.6	27.5	60.1	27.1	9.0	45.4	**4.9**	**1.0**	**5.7**

Table 4 gives the main result; all models, all attacks and the retained share of the cohort are in Supplementary Tables S8–S10. The old defenses do not survive the adaptive attackers. MPS and LGAT leave 33–100% white-box Real ASR on every corpus and library, and LGAT 99.5–100% on CSE-CIC-IDS2018 and for XGBoost on CIC-IDS2017; its protection against FFE-v1

collapses when the budget moves from the training value $M = 5$ to $M = 20$ (XGBoost on CSE-CIC-IDS2018: 25.6% to 73.1%).

V2 changes this where the certificate applies. On CIC-IDS2017 its Real ASR is at most 0.7% against every attacker and library, with recall higher than the undefended model by 0.07–0.46 point; the stand-alone gate flags at least 99.8% of the cohort. Counted over the whole cohort, V2 keeps at least 99.2% of the CIC-IDS2017 flows detected under every attacker, against 0–66% for the undefended models. On CSE-CIC-IDS2018 it is 0.0–0.3% against CAPGD and 0.0–8.1% against FFE-v1, but 8.8%, 7.9% and 8.1% against the white-box attacker (CatBoost, LightGBM, XGBoost), compared with 99.5–100% for the undefended models and for LGAT. The remaining 8% are cohort flows that the gate does not flag and the free classifier loses under attack; V2 keeps at least 91% of the whole cohort detected under every attack, so the residual is a coverage gap of the gate, not an attack on the certificate. Free classifiers alone stay about 80% evadable by the white-box attacker on CSE-CIC-IDS2018; the gate closes that gap. Accuracy changes by between -0.38 and $+0.08$ point on every cell.

UNSW-NB15, with overlapping classes and about 88% clean accuracy, is the hard case. At strict parity V2 leaves 12.2%, 22.0% and 4.9% FFE-v1 and 25.4%, 31.9% and 5.7% white-box Real ASR for XGBoost, LightGBM and CatBoost, against 24–91% undefended; CAPGD reaches 1.0–2.1%. It is not uniformly better than the older defense: LGAT is clearly better against FFE-v1 for LightGBM (14.5% against 22.0% at $M = 20$) and indistinguishable for XGBoost at $M = 5$ ($p = 0.86$), while V2 is better against the white-box attacker and CAPGD. The share of the cohort retained under attack is only

67–94%. The relaxed tier V2-R, which gives up 2.1–2.4 accuracy points and gains 6.4–7.3 recall points relative to the undefended model, reaches 0.5–4.7% against every attacker and retains 95–98%, so on this corpus robustness costs accuracy.

Price of the certificate.. The stand-alone gate loses about 0.7 accuracy, 3.7–3.8 F1 and 5.4–5.8 recall points on CSE-CIC-IDS2018, 1.6–1.8 accuracy points on UNSW-NB15 for XGBoost and LightGBM, and nothing on CIC-IDS2017; it misses 0.1–9% of the cohort that the undefended model flags (Supplementary Table S7), and its Real ASR, like every entry of Table 4, is computed on the flows still detected, so a 0.0% for the gate is not complete coverage. The gate has 300 trees of depth 8 against 60 of depth 6 for the undefended model, and a capacity-matched unconstrained arm is not yet available, so the accuracy cost is confounded with capacity *[placeholder: follow-up pending]*. Fusion with the free classifier recovers clean parity. Building V2 takes 5–33 times the undefended training time and inference runs two ensembles.

What is not shown.. FFE-v2 was not evaluated against the hardened models; random forests (supported by the recipe) and the manipulation-signature detector were not evaluated; the white-box attacker is greedy single-feature threshold crossing, so a stronger search could raise the residual of the free component but not of the gate; validation used a 300-flow cohort and a small grid; and the monotone direction is a modelling assumption that removes the ability to learn "short and small is malicious" patterns, which is the source of the accuracy cost. Beyond Proposition 2 we claim no robustness for unflagged flows.

8. Discussion and Conclusions

8.1. What the Results Support, and What They Do Not

The attack.. With score access alone, FFE-v2 matches or exceeds our greedy threshold-crossing control in nine of nine corpus-library cells of a single frozen evaluation (significantly in four) and is above FFE-v1 in seven; the five-seed means point the same way (Tables 2 and 3). The gain over FFE-v1 comes from repairing diagnosed weaknesses, in particular the window target and the missing payload-free packet moves, at a price of 28–38 queries (109 on CIC-IDS2017 CatBoost), more than FFE-v1 on the easy corpora and far more than the control (2) or CAPGD (about 3 with an uncounted surrogate). The result is conditional on detection, on the 12-feature schema and on a cost model that FFE steers by. Three limits matter. First, a cell where all methods agree is not an attack success: CatBoost on CIC-IDS2017 (36.4%) measures how much that model leans on immutable features. Second, the cost story is mixed: on UNSW-NB15 FFE-v2's evasions need more operational cost than FFE-v1's on the flows both evade. Third, the protocol (validation tuning, frozen configuration, one test evaluation) is not a clean hold-out, since the failures that motivated FFE-v2 were seen on the test cohorts, the cohorts are single draws, and the five-seed study, though cleaner, used a configuration tuned on the main split. The claim that FFE-v2 is the strongest attack here is limited to the attacks we ran: the control is our own implementation and is erratic across seeds on CIC-IDS2017 LightGBM (95.7%, 41.0%, 95.7%), CAPGD was run on XGBoost in the single-split test and on three seeds of the other libraries, and LT-Attack and Cascioli et al. were not run.

The defense.. The certificate is conditional and narrow. If the port, protocol and backward features stay fixed and the four increase-only columns do not decrease, every flow the monotone gate flags stays flagged; because the gate reads the backward features without being monotone in them, and segmentation or chaff may change server responses, this is not a worst-case bound on an attack, and a lower-footprint variant of the same attack flow is not covered. The stand-alone gate's 0.0% Real ASR is computed on the flows it still detects and is not a measure of coverage. The residual is about 8% of the CSE-CIC-IDS2018 cohort for all three libraries and 5–32% of UNSW-NB15, and the free classifier that covers it carries no guarantee. The gate costs accuracy where "short and small" flows are informative (0.7 points on CSE-CIC-IDS2018, 1.6–1.8 on UNSW-NB15), confounded with its larger capacity until a capacity-matched unconstrained arm is run; fusion recovers it on CSE-CIC-IDS2018 but only partly on UNSW-NB15, where LGAT remains better against FFE-v1 for LightGBM. The free classifiers were trained against FFE-v1 and the white-box attacker and are evaluated with the same attack code. The most important missing experiment is FFE-v2 against the hardened models; random forests, a stronger adaptive attacker against the fusion rule and more than one seed are also open.

Feasibility and validity.. "Feasible" means conformant to the 12-feature schema verifier and, for FFE-v2, replayed on synthetic traces (Section 6). The schema holds backward packets fixed, which is exact for pure-ACK chaff but optimistic for re-segmentation; the accounting of chaff bytes decides whether 100% or 69.2% of the UNSW-NB15 evasions survive replay; payload semantics were not tested; and a deployed detector may read fea-

tures outside the twelve. The defense inherits these assumptions. Cohorts contain only flows that the models detect when clean (77.6% of the CSE-CIC-IDS2018 test attacks), MOEVA and PSO ran on 200 flows, CAPGD and the NIDSGAN-style generator are our re-implementations with one small surrogate, and the random split lets near-duplicates fall on both sides so accuracies are optimistic.

8.2. Concluding Remarks and Future Directions

This article measured, at full scale and across three tree libraries, how far a schema-constrained attacker with score access can evade flow-based NIDS, and what a defender can promise in return. A diagnosis of FFE-v1 on validation data led to FFE-v2: a re-segmentation generator that preserves the Lie-cone invariants, multi-scale probing, window targets from the attacker's benign sample, log-odds progress and cost refinement. In one frozen evaluation (not a clean hold-out) FFE-v2 reaches 99.5%, 100.0% and 98.3% Real ASR on XGBoost, equals or exceeds our greedy white-box control in all nine cells and repairs the LightGBM and CatBoost failures of FFE-v1, with 28–38 queries and a higher operational cost than FFE-v1 on UNSW-NB15; CatBoost on CIC-IDS2017 is capped at 36.4% by immutable features. OR-fusing a monotone gate with a free classifier (hardened on five of nine cells) cuts Real ASR on the flows still detected, against FFE-v1, CAPGD and a greedy white-box attacker, from 5–100% to at most 8.8% on CSE-CIC-IDS2018 and CIC-IDS2017 and to 1.0–32% on UNSW-NB15, with a conditional guarantee for flagged flows. Future work includes FFE-v2 against the hardened models, replay on a real network stack including server ACK behavior, the official white-box tree attacks, random forests and neural detectors, time- or

host-grouped splits, and further public corpora.

CRediT authorship contribution statement

Quang-Vinh Dang: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no external funding.

Data availability

The code, the dataset builders and the per-flow result archives for FFE-v1, FFE-v2, the white-box control, all baselines (naive, ZOO-NIDS, HopSkipJump-NIDS, MOEVA, PSO, CAPGD, NIDSGAN-style), the validation tuning, the multi-seed runs, the packet replay and the defenses are at https://github.com/vinhqdang/adversarial_ids; every table is regenerated from the result files by `experiments/summarize_results.py`. All flow data come from the public CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15 archives.

References

[1] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116. doi:10.5220/0006639801080116.

[2] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794. doi:10.1145/2939672.2939785.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, LightGBM: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 30, 2017, pp. 3146–3154, neurIPS proceedings carry no DOI.
URL https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, A. Gulin, CatBoost: Unbiased boosting with categorical features, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 31, 2018, pp. 6638–6648. doi:10.48550/arXiv.1706.09516.

[5] Q.-V. Dang, Using machine learning for intrusion detection systems, Computing and Informatics 41 (1) (2022) 12–33. doi:10.31577/cai_2022_1_12.

[6] Q.-V. Dang, Studying the attack detection problem using the dataset CIDDS-001, in: Digital Science (DSIC 2021), Vol. 381 of Lecture Notes in Networks and Systems, Springer, Cham, 2022, pp. 525–532. doi:10.1007/978-3-030-93677-8_46.

[7] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, M. Colajanni, Modeling realistic adversarial attacks against network intrusion detection systems, ACM Digital Threats: Research and Practice 3 (3) (2022) 1–19. doi:10.1145/3469659.

[8] R. Sommer, V. Paxson, Outside the closed world: On using machine learning for network intrusion detection, in: Proceedings of the 2010 IEEE Symposium on Security and Privacy (S&P), 2010, pp. 305–316. doi:10.1109/SP.2010.25.

[9] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, F. A. Wichmann, Shortcut learning in deep neural networks, Nature Machine Intelligence 2 (11) (2020) 665–673. doi:10.1038/s42256-020-00257-z.

[10] F. Pierazzi, F. Pendlebury, J. Cortellazzi, L. Cavallaro, Intriguing properties of adversarial ML attacks in the problem space, in: Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P), 2020, pp. 1332–1349. doi:10.1109/SP40000.2020.00073.

[11] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, C.-J. Hsieh, ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models, in: Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security (AISEC), 2017, pp. 15–26. doi:10.1145/3128572.3140448.

[12] J. Chen, M. I. Jordan, M. J. Wainwright, HopSkipJumpAttack: A query-efficient decision-based attack, in: Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P), 2020, pp. 1277–1294. doi:10.1109/SP40000.2020.00045.

[13] T. Simonetto, S. Ghamizi, M. Cordy, Constrained adaptive attack: Effective adversarial attack against deep neural networks for tabular data, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 27817–27849. doi:10.52202/079017-0873.

[14] S. Ghamizi, M. Cordy, M. Papadakis, Y. L. Traon, Search-based adversarial testing and improvement of constrained credit scoring systems, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2020, pp. 1089–1100. doi:10.1145/3368089.3409739.

[15] D. Han, Z. Wang, Y. Zhong, W. Chen, J. Yang, S. Lu, X. Shi, X. Yin, Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors, IEEE Journal on Selected Areas in Communications 39 (8) (2021) 2632–2647. doi:10.1109/JSAC.2021.3087242.

[16] B.-E. Zolbayar, R. Sheatsley, P. McDaniel, M. J. Weisman, S. Zhu, S. Zhu, S. Krishnamurthy, Generating practical adversarial network traffic flows using NIDSGAN, arXiv preprint (2022). arXiv:2203.06694, doi:10.48550/arXiv.2203.06694.

[17] C. Zhang, H. Zhang, C.-J. Hsieh, An efficient adversarial attack for tree ensembles, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 16165–16176. doi:10.48550/arXiv.2010.11598.

[18] Í. Í. Romeo, M. Theodorides, S. Afroz, D. Wagner, Adversarially robust malware detection using monotonic classification, in: Proceedings of the Fourth ACM International Workshop on Security and Privacy Analytics (IWSPA), 2018, pp. 54–63. doi:10.1145/3180445.3180449.

[19] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, R. Fergus, Intriguing properties of neural networks, in: Proceedings of the International Conference on Learning Representations (ICLR), 2014. doi:10.48550/arXiv.1312.6199.

[20] I. J. Goodfellow, J. Shlens, C. Szegedy, Explaining and harnessing adversarial examples, in: International Conference on Learning Representations (ICLR), 2015. doi:10.48550/arXiv.1412.6572.

[21] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, A. Swami, The limitations of deep learning in adversarial settings, in: Proceedings of the 2016 IEEE European Symposium on Security and Privacy (EuroS&P), 2016, pp. 372–387. doi:10.1109/EuroSP.2016.36.

[22] N. Carlini, D. Wagner, Towards evaluating the robustness of neural networks, in: Proceedings of the 38th IEEE Symposium on Security and Privacy (S&P), 2017, pp. 39–57. doi:10.1109/SP.2017.49.

[23] G. Apruzzese, H. S. Anderson, L. Cavallaro, "Real attackers don't compute gradients": Bridging the gap between adversarial ML research and practice in cybersecurity, in: Proceedings of the 1st IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), 2023, pp. 339–364. doi:10.1109/SaTML54575.2023.00031.

[24] K. He, D. D. Kim, M. R. Asghar, Adversarial machine learning for network intrusion detection systems: A comprehensive survey, IEEE Communications Surveys & Tutorials 25 (1) (2023) 538–566. doi:10.1109/COMST.2022.3233793.

[25] M. A. Merzouk, F. Cuppens, N. Boulahia-Cuppens, R. Yaich, Investigating the practicality of adversarial evasion attacks on network intrusion detection, Annals of Telecommunications 77 (2022) 763–775. doi:10.1007/s12243-022-00910-1.

[26] R. Sheatsley, N. Papernot, M. Weisman, G. Verma, P. McDaniel, Adversarial examples for network intrusion detection systems, Journal of Computer Security 30 (2022) 727–752. doi:10.3233/JCS-210094.

[27] I. Debicha, B. Cochez, T. Kenaza, T. Debatty, J.-M. Dricot, W. Mees, Adv-Bot: Realistic adversarial botnet attacks against network intrusion detection systems, Computers & Security 129 (2023) 103176. doi:10.1016/j.cose.2023.103176.

[28] H. Zhang, D. Han, Y. Liu, Z. Wang, J. Sun, S. Zhuang, J. Liu, J. Dong, Explainable and transferable adversarial attack for ML-based network intrusion detectors, IEEE Transactions on Dependable and Secure Computing 22 (2025) 5090–5107. doi:10.1109/TDSC.2025.3560486.

[29] M. elShehaby, A. Matrawy, A novel perturb-ability score to mitigate evasion adversarial attacks on flow-based ML-NIDS, Journal of Information Security and Applications 99 (2026) 104409. doi:10.1016/j.jisa.2026.104409.

[30] Z. Lin, Y. Shi, Z. Xue, IDSGAN: Generative adversarial networks for attack generation against intrusion detection, in: Advances in Knowledge Discovery and Data

Mining (PAKDD), Lecture Notes in Computer Science, 2022, pp. 79–91. doi:10.1007/978-3-031-05981-0_7.

[31] K. Kireev, B. Kulynych, C. Troncoso, Adversarial robustness for tabular data through cost and utility awareness, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), 2023. doi:10.14722/ndss.2023.24924.

[32] T. Simonetto, S. Ghamizi, M. Cordy, TabularBench: Benchmarking adversarial robustness for tabular deep learning in real-world use-cases, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 78394–78430. doi:10.52202/079017-2492.

[33] Z. He, C. Ouyang, L. Wen, C. Liu, C. Moreira, TabAttackBench: A benchmark for adversarial attacks on tabular data, Expert Systems with Applications 301 (2026) 130491. doi:10.1016/j.eswa.2025.130491.

[34] L. Cascioli, L. Devos, O. Kuželka, J. Davis, Faster repeated evasion attacks in tree ensembles, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 24762–24794. doi:10.52202/079017-0780.

[35] L. Devos, L. Perini, W. Meert, J. Davis, Detecting evasion attacks in deployed tree ensembles, in: Machine Learning and Knowledge Discovery in Databases: Research Track (ECML PKDD 2023), Lecture Notes in Computer Science, 2023, pp. 120–136. doi:10.1007/978-3-031-43424-2_8.

[36] D. Vos, S. Verwer, Robust optimal classification trees against adversarial examples, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 36, 2022, pp. 8520–8528. doi:10.1609/aaai.v36i8.20829.

[37] B. Lindqvist, On the adversarial robustness of decision trees and a symmetry defense, IEEE Access 13 (2025) 16120–16132. doi:10.1109/ACCESS.2025.3530695.

[38] A. Kantchelian, J. D. Tygar, A. Joseph, Evasion and hardening of tree ensemble classifiers, in: Proceedings of the 33rd International Conference on Machine Learning (ICML), 2016, pp. 2387–2396. doi:10.48550/arXiv.1509.07892.

[39] A. Ignatiev, Y. Izza, P. J. Stuckey, J. Marques-Silva, Using MaxSAT for efficient explanations of tree ensembles, in: Proceedings of the 36th AAAI Conference on Artificial Intelligence, Vol. 36, 2022, pp. 3776–3785. doi:10.1609/aaai.v36i4.20292.

[40] Y. Izza, A. Ignatiev, J. Marques-Silva, On tackling explanation redundancy in decision trees, Journal of Artificial Intelligence Research 75 (2022) 261–321. doi:10.1613/jair.1.13575.

[41] H. Chen, H. Zhang, S. Si, Y. Li, D. Boning, C.-J. Hsieh, Robust decision trees against adversarial examples, in: Proceedings of the 36th International Conference on Machine Learning (ICML), 2019, pp. 1122–1131. doi:10.48550/arXiv.1902.10660.

[42] M. Andriushchenko, M. Hein, Provably robust boosted decision stumps and trees against adversarial attacks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 12997–13008. doi:10.48550/arXiv.1906.03526.

[43] S. Calzavara, C. Lucchese, G. Tolomei, S. A. Abebe, S. Orlando, Treant: training evasion-aware decision trees, Data Mining and Knowledge Discovery 34 (5) (2020) 1390–1420. doi:10.1007/s10618-020-00694-9.

[44] D. Vos, S. Verwer, Efficient training of robust decision trees against adversarial examples, in: Proceedings of the 38th International Conference on Machine Learning

(ICML), Vol. 139 of Proceedings of Machine Learning Research, 2021, pp. 10586–10595. doi:10.48550/arXiv.2012.10438.

[45] M. J. Hashemi, E. Keller, Enhancing robustness against adversarial examples in network intrusion detection systems, in: Proceedings of the 2020 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN), 2020, pp. 37–43. doi:10.1109/NFV-SDN50289.2020.9289869.

[46] S. Calzavara, L. Cazzaro, G. E. Pibiri, N. Prezza, Verifiable learning for robust tree ensembles, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2023, pp. 1850–1864. doi:10.1145/3576915.3623100.

[47] S. Calzavara, L. Cazzaro, C. Lucchese, G. E. Pibiri, Verifiable boosted tree ensembles, in: 2025 IEEE Symposium on Security and Privacy (SP), 2025, pp. 2057–2075. doi:10.1109/SP61157.2025.00022.

[48] J. Postel, Internet protocol, RFC 791, RFC Editor (1981). doi:10.17487/RFC0791.

[49] W. Eddy, Transmission control protocol (TCP) specification, RFC 9293, RFC Editor (2022). doi:10.17487/RFC9293.

[50] J. Postel, User datagram protocol, RFC 768, RFC Editor (1980). doi:10.17487/RFC0768.

[51] T. Ylonen, C. Lonvick, The secure shell (SSH) transport layer protocol, RFC 4253, RFC Editor (2006). doi:10.17487/RFC4253.

[52] D. Borman, B. Braden, V. Jacobson, R. Scheffenegger, TCP extensions for high performance, RFC 7323, RFC Editor (2014). doi:10.17487/RFC7323.

[53] C. Carathéodory, Vorlesungen über reelle Funktionen, B. G. Teubner, Leipzig, 1918.

[54] M. Nagumo, Über die Lage der Integralkurven gewöhnlicher Differentialgleichungen, Proceedings of the Physico-Mathematical Society of Japan. 3rd Series 24 (1942) 551–559. doi:10.11429/ppmsj1919.24.0_551.

[55] H. Brézis, On a characterization of flow-invariant sets, Communications on Pure and Applied Mathematics 23 (2) (1970) 261–263. doi:10.1002/cpa.3160230211.

[56] M. Cantone, C. Marrocco, A. Bria, Machine learning in network intrusion detection: A cross-dataset generalization study, IEEE Access 12 (2024) 144489–144508. doi:10.1109/ACCESS.2024.3472907.

[57] N. Moustafa, J. Slay, UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS), 2015, pp. 1–6. doi:10.1109/MilCIS.2015.7348942.

[58] G. Engelen, V. Rimmer, W. Joosen, Troubleshooting an intrusion detection dataset: the CICIDS2017 case study, in: 2021 IEEE Security and Privacy Workshops (SPW), 2021, pp. 7–12. doi:10.1109/SPW53761.2021.00009.

[59] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, E. Totel, Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes, in: Risks and Security of Internet and Systems (CRiSIS 2022), Lecture Notes in Computer Science, 2023, pp. 18–33. doi:10.1007/978-3-031-31108-6_2.